

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import roc_auc_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report
print("Step 1: Libraries imported successfully.")
```

Step 1: Libraries imported successfully.

```
try:
    # Step 2: Load the dataset and perform initial data cleaning
    df = pd.read_csv("/content/RSK project data.csv")
    print("\nStep 2: Data Loading and Cleaning...")
    df_cleaned = df.copy()

    # Convert 'QUARTER-END-DATE' to datetime objects
    df_cleaned['QUARTER-END-DATE'] = pd.to_datetime(df_cleaned['QUARTER-END-DATE'], dayfirst=True, errors='coerce')

    # Define columns to clean (remove '%' and ',' and convert to numeric)
    cols_to_clean = ['TOTAL ADVANCES', 'GROSS NPA', 'GNPA RATIO', 'PROVISION RATE', 'PROVISION',
                     'NET NPA', 'NPA RATIO', 'RECOVERY', 'NO.OF ACCOUNTS', 'REPO RATES', 'CPI', 'GDP']

    for col in cols_to_clean:
        if col in df_cleaned.columns:
            df_cleaned[col] = df_cleaned[col].astype(str).str.replace('%', '').str.replace(',', '').str.strip()
            df_cleaned[col] = pd.to_numeric(df_cleaned[col], errors='coerce')

    # Sort data by 'LOAN TYPE' and 'QUARTER-END-DATE'
    df_cleaned = df_cleaned.sort_values(by=['LOAN TYPE', 'QUARTER-END-DATE']).reset_index(drop=True)
    print("Data cleaned and sorted.")
    print("Cleaned data has {} rows and {} columns.".format(df_cleaned.shape[0], df_cleaned.shape[1]))

    # Display the first and last few rows of the cleaned DataFrame only if loading was successful
    print("First 5 rows of the DataFrame:")
    display(df_cleaned.head())
    print("\nLast 5 rows of the DataFrame:")
    display(df_cleaned.tail())
except Exception as e:
    print(f"An error occurred: {e}")
```

Step 2: Data Loading and Cleaning...

Data cleaned and sorted.

Cleaned data has 100 rows and 15 columns.

First 5 rows of the DataFrame:

	LOAN TYPE	QUARTER-END-DATE	QUARTERS	TOTAL ADVANCES	GROSS NPA	GNPA RATIO	PROVISION RATE	PROVISION	NET NPA	NPA RATIO	RECOVERY	NO.OF ACCOUNTS
0	AGRICULTURE	2019-03-31	Q1 FY19	6548273.0	506171.0	7.73	71.0	359381.41	146789.59	2.24	280034.0	
1	AGRICULTURE	2019-06-30	Q2 FY19	8713332.0	930142.0	10.67	71.0	660400.82	269741.18	3.10	212550.0	
2	AGRICULTURE	2019-09-30	Q3 FY19	3962509.0	1879384.0	47.43	70.0	1315568.80	563815.20	14.23	787767.0	
3	AGRICULTURE	2019-12-31	Q4 FY19	5648772.0	264939.0	4.69	68.0	180158.52	84780.48	1.50	125797.0	
4	AGRICULTURE	2020-03-31	Q1 FY20	3369719.0	728726.0	21.63	72.0	524682.72	204043.28	6.06	503748.0	

Last 5 rows of the DataFrame:

	LOAN TYPE	QUARTER-END-DATE	QUARTERS	TOTAL ADVANCES	GROSS NPA	GNPA RATIO	PROVISION RATE	PROVISION	NET NPA	NPA RATIO	RECOVERY	NO.OF ACCOUNTS
95	SME	2022-12-31	Q4 FY22	7927958.0	1846679.0	23.29	73.0	1348075.67	498603.33	6.29	472490.0	14
96	SME	2023-03-31	Q1 FY23	10267018.0	7814071.0	40.25	72.0	5704008.82	2110043.17	10.00	2570021.0	16

```
# Step 3: Define the Target Variable
print("\nStep 3: Defining the Target Variable...")
# Calculate the maximum GNPA ratio in the next 4 quarters for each loan type
df_cleaned['GNPA_MAX_NEXT_4Q'] = df_cleaned.groupby('LOAN TYPE')['GNPA RATIO'].shift(-4).rolling(window=4, min_p
# Define the target variable: 1 if GNPA ratio deteriorates by 0.5% or more, 0 otherwise
df_cleaned['TARGET_DETERIORATION'] = (df_cleaned['GNPA_MAX_NEXT_4Q'] - df_cleaned['GNPA RATIO'] >= 0.5).astype(i

# Create a copy of the dataframe for modeling, dropping rows with NaNs in target related columns
```

```
df_model_data = df_cleaned.dropna(subset=['TARGET_DETERIORATION', 'GNPA_MAX_NEXT_4Q']).copy()
print("Target variable 'TARGET_DETERIORATION' created.")
print("Value counts for the target variable:")
print(df_model_data['TARGET_DETERIORATION'].value_counts())
```

```
Step 3: Defining the Target Variable...
Target variable 'TARGET_DETERIORATION' created.
Value counts for the target variable:
TARGET_DETERIORATION
1      76
0     19
Name: count, dtype: int64
```

```
# Step 4: Feature Engineering
print("\nStep 4: Engineering Features...")
# Create a copy to avoid SettingWithCopyWarning
df_model_data = df_model_data.copy()

# Create lagged GNPA Ratio features
df_model_data['GNPA_RATIO_lag1'] = df_model_data.groupby('LOAN TYPE')['GNPA_RATIO'].shift(1)
df_model_data['GNPA_RATIO_lag2'] = df_model_data.groupby('LOAN TYPE')['GNPA_RATIO'].shift(2)

# Create rolling mean of TOTAL ADVANCES (lagged by 1 quarter)
df_model_data['TOTAL_ADVANCES_rollmean4'] = df_model_data.groupby('LOAN TYPE')['TOTAL ADVANCES'].shift(1).rolling(4).mean()

# Calculate Quarter-over-Quarter growth for TOTAL ADVANCES
df_model_data['ADV_QOQ_GROWTH'] = df_model_data.groupby('LOAN TYPE')['TOTAL ADVANCES'].pct_change()

# Create lagged economic factor features
economic_factors = ['REPO_RATES', 'CPI', 'GDP']
for factor in economic_factors:
    df_model_data[f'{factor}_lag1'] = df_model_data.groupby('LOAN TYPE')[factor].shift(1)

# Drop rows with NaN values introduced by lagging and rolling operations
df_model_data = df_model_data.dropna(subset=['GNPA_RATIO_lag1', 'GNPA_RATIO_lag2', 'TOTAL_ADVANCES_rollmean4'])
print("Feature engineering complete.")
```

```
Step 4: Engineering Features...
Feature engineering complete.
```

```
print("\nStep 5: Preparing Data for Modeling...")

# Define the features to be used in the model
features = [
    'GNPA_RATIO_lag1', 'GNPA_RATIO_lag2',
    'TOTAL_ADVANCES_rollmean4', 'ADV_QOQ_GROWTH',
    'REPO_RATES_lag1', 'CPI_lag1', 'GDP_lag1']

# Separate features (X) and target (y)
X = df_model_data[features]
y = df_model_data['TARGET_DETERIORATION']

# Determine the split date for time-series based train-test split
# Taking the 5th unique quarter-end date from the end for the split
split_date = df_model_data['QUARTER-END-DATE'].unique()[-5]

# Split data into training and testing sets based on the split_date
train_df = df_model_data[df_model_data['QUARTER-END-DATE'] < split_date]
test_df = df_model_data[df_model_data['QUARTER-END-DATE'] >= split_date]

X_train, y_train = train_df[features], train_df['TARGET_DETERIORATION']
X_test, y_test = test_df[features], test_df['TARGET_DETERIORATION']

# Initialize and fit SimpleImputer to fill missing values (e.g., from lagged features)
imputer = SimpleImputer(strategy='median')
X_train = pd.DataFrame(imputer.fit_transform(X_train), columns=features)
X_test = pd.DataFrame(imputer.transform(X_test), columns=features)

print("Data split into training and testing sets. Missing values imputed.")
print(f"Training data shape: {X_train.shape}")
print(f"Testing data shape: {X_test.shape}")
```

```
Step 5: Preparing Data for Modeling...
Data split into training and testing sets. Missing values imputed.
Training data shape: (60, 7)
Testing data shape: (25, 7)
```

```
print("\nStep 6: Training the Random Forest Model...")
# Initialize RandomForestClassifier with specified parameters
```

```
# n_estimators: number of trees in the forest
# random_state: for reproducibility
# class_weight='balanced': handles imbalanced classes by adjusting weights
# max_depth: maximum depth of the tree
rf_model = RandomForestClassifier(n_estimators=200, random_state=42, class_weight='balanced_subsample', max_depth=10)
rf_model.fit(X_train, y_train)
print("Model training complete.")
```

Step 6: Training the Random Forest Model...
Model training complete.

```
# Step 7: Evaluate the Model
print("\nStep 7: Evaluating the Model...")
# Make predictions on the test set
preds = rf_model.predict(X_test)
# Get probabilities for the positive class
probs = rf_model.predict_proba(X_test)[:, 1]

# Print various evaluation metrics
print(f"AUC-ROC Score: {roc_auc_score(y_test, probs):.4f}")
print(f"Precision Score: {precision_score(y_test, preds):.4f}")
print(f"Recall Score: {recall_score(y_test, preds):.4f}")
print(f"F1 Score: {f1_score(y_test, preds):.4f}")

print("\nClassification Report:\n", classification_report(y_test, preds))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, preds))

# Visualize the Confusion Matrix
cm = confusion_matrix(y_test, preds)
plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted No Deterioration', 'Predicted Deterioration'],
            yticklabels=['Actual No Deterioration', 'Actual Deterioration'])
plt.title('Model Prediction Accuracy (Confusion Matrix)', fontsize=16)
plt.ylabel('Actual Outcome')
plt.xlabel('Predicted Outcome')
plt.show()
print("Confusion Matrix plot generated.")
```

Step 7: Evaluating the Model...

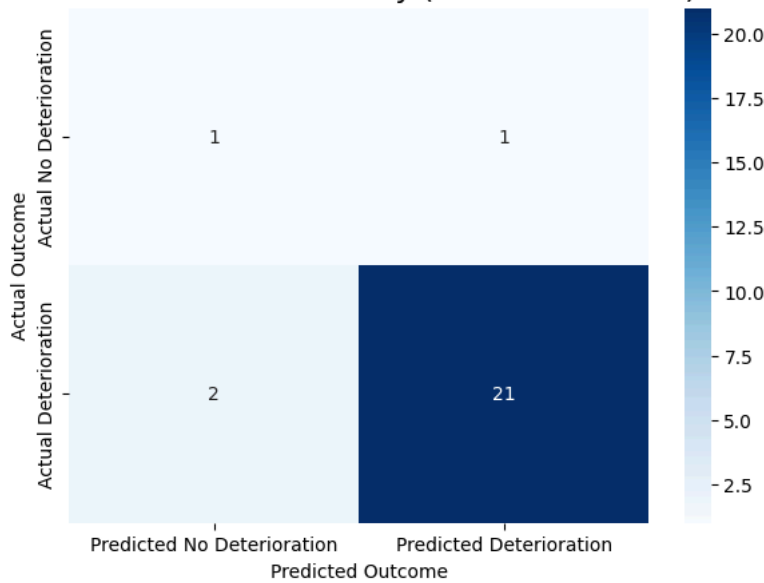
AUC-ROC Score: 0.9130
 Precision Score: 0.9545
 Recall Score: 0.9130
 F1 Score: 0.9333

Classification Report:	precision	recall	f1-score	support
0	0.33	0.50	0.40	2
1	0.95	0.91	0.93	23
accuracy			0.88	25
macro avg	0.64	0.71	0.67	25
weighted avg	0.90	0.88	0.89	25

Confusion Matrix:

```
[[ 1  1]
 [ 2 21]]
```

Model Prediction Accuracy (Confusion Matrix)

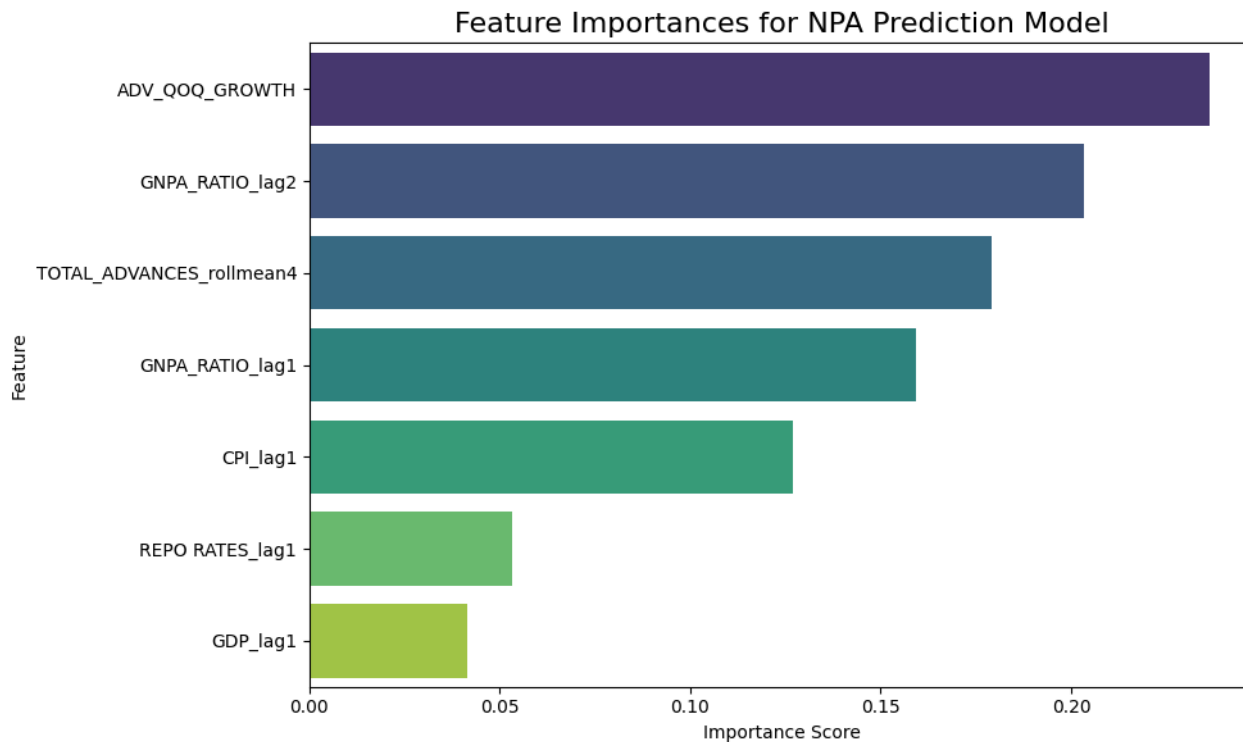


Confusion Matrix plot generated.

```
print("\nStep 8: Analyzing Feature Importances...")
# Create a DataFrame for feature importances
importance_df = pd.DataFrame({
    'feature': features,
    'importance': rf_model.feature_importances_
}).sort_values('importance', ascending=False)

# Plot feature importances
plt.figure(figsize=(10, 6))
sns.barplot(x='importance', y='feature', data=importance_df, palette='viridis', hue='feature', legend=False)
plt.title('Feature Importances for NPA Prediction Model', fontsize=16)
plt.xlabel('Importance Score')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()
print("Feature importance plot generated.")
```

Step 8: Analyzing Feature Importances...



Feature importance plot generated.

```
print("Displaying the `importance_df` DataFrame to confirm feature importances:")
display(importance_df)
```

```
print("\nTop 5 Most Important Features:")
for index, row in importance_df.head(5).iterrows():
    print(f"{index} - {row['feature']}: {row['importance']:.4f}")
```

Displaying the `importance\_df` DataFrame to confirm feature importances:

	feature	importance	
3	ADV_QOQ_GROWTH	0.236314	
1	GNPA_RATIO_lag2	0.203501	
2	TOTAL_ADVANCES_rollmean4	0.179121	
0	GNPA_RATIO_lag1	0.159167	
5	CPI_lag1	0.127019	
4	REPO_RATES_lag1	0.053288	
6	GDP_lag1	0.041590	

Top 5 Most Important Features:

```
- ADV_QOQ_GROWTH: 0.2363
- GNPA_RATIO_lag2: 0.2035
- TOTAL_ADVANCES_rollmean4: 0.1791
- GNPA_RATIO_lag1: 0.1592
- CPI_lag1: 0.1270
```

Next steps: [Generate code with importance_df](#) [New interactive sheet](#)

```
# Step 9: Generate Final Predictions for Latest Data
print("\nStep 9: Generating Final Predictions...")
# 1. Use the most recent data point for each Loan Type
latest_data = df_model_data.loc[df_model_data.groupby('LOAN TYPE')['QUARTER-END-DATE'].idxmax()]

# 2. Transform features using the fitted imputer to maintain consistency with training
# We pass it as a DataFrame to avoid feature name/order issues
X_latest = pd.DataFrame(imputer.transform(latest_data[features]), columns=features)

# 3. Predict probabilities (avoiding .values to keep metadata intact if needed)
latest_probs = rf_model.predict_proba(X_latest)[: , 1]

# 4. Create final DataFrame
final_predictions = pd.DataFrame({
```

```

'LOAN TYPE': latest_data['LOAN TYPE'].values,
'RISK_PROBABILITY': latest_probs
}).sort_values('RISK_PROBABILITY', ascending=False).reset_index(drop=True)

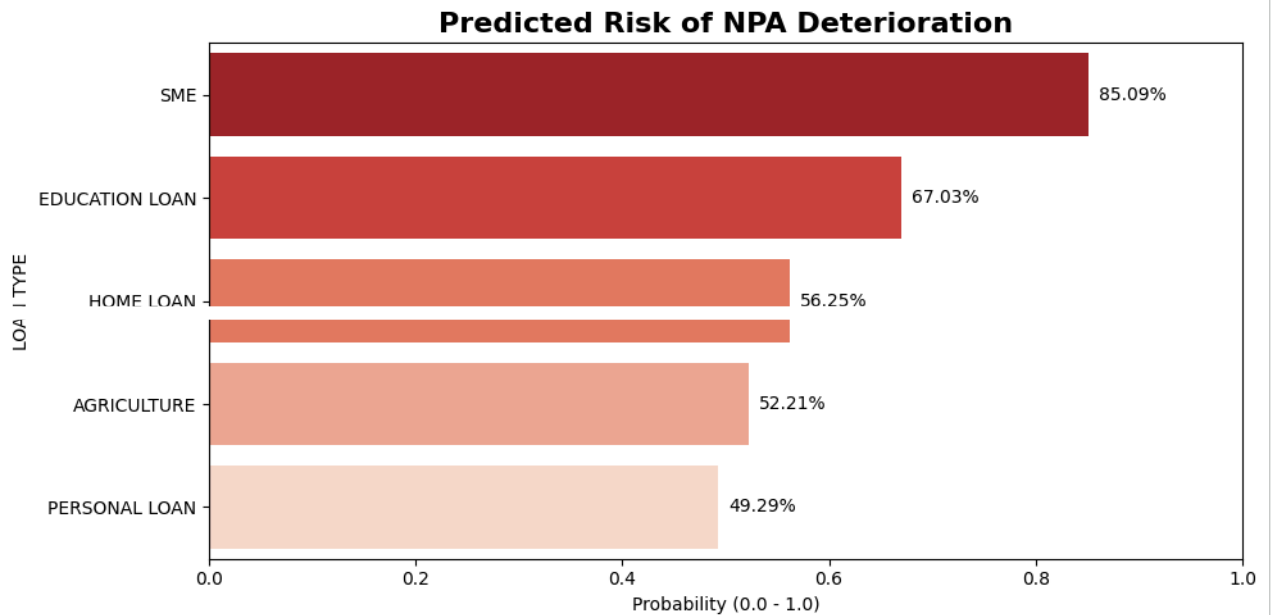
# 5. Visualization with fixed logic
plt.figure(figsize=(10, 5))
sns.barplot(x='RISK_PROBABILITY', y='LOAN TYPE', data=final_predictions, palette='Reds_r', hue='LOAN TYPE', legend=True)
plt.title('Predicted Risk of NPA Deterioration', fontsize=16, weight='bold')
plt.xlabel('Probability (0.0 - 1.0)')
plt.xlim(0, 1)

for i, prob in enumerate(final_predictions['RISK_PROBABILITY']):
    plt.text(prob + 0.01, i, f"{prob:.2%}", va='center')

plt.tight_layout()
plt.show()
display(final_predictions)

```

Step 9: Generating Final Predictions...



	LOAN TYPE	RISK_PROBABILITY	
0	SME	0.850937	
1	EDUCATION LOAN	0.670322	
2	HOME LOAN	0.562524	
3	AGRICULTURE	0.522140	
4	PERSONAL LOAN	0.492900	

Next steps: [Generate code with final_predictions](#) [New interactive sheet](#)

```

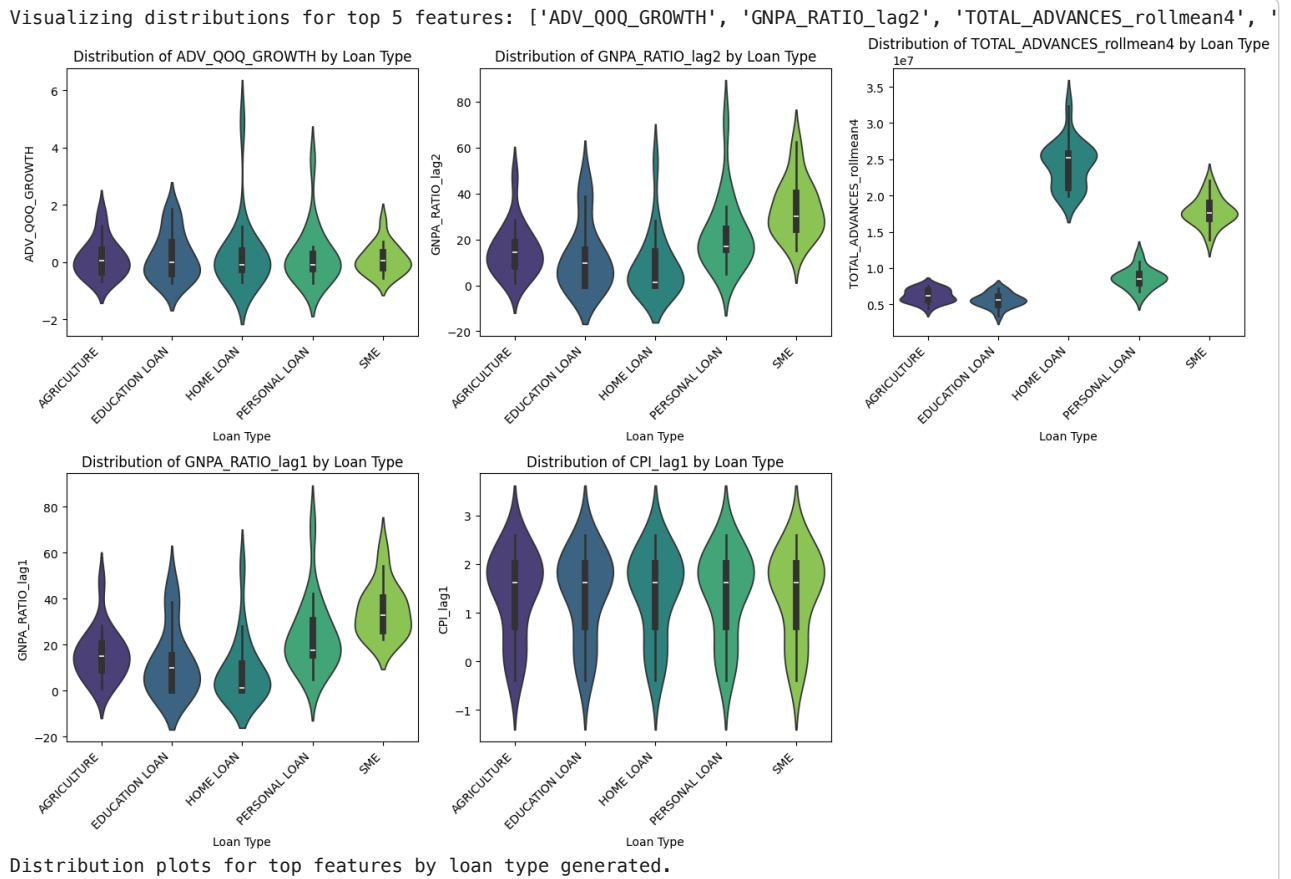
# Get the top 5 important features from the importance_df
top_features = importance_df['feature'].head(5).tolist()

print(f"Visualizing distributions for top 5 features: {top_features}")

# Create violin plots for each top feature by LOAN TYPE
plt.figure(figsize=(15, 10))
for i, feature in enumerate(top_features):
    plt.subplot(2, 3, i + 1) # Create a 2x3 grid of subplots
    sns.violinplot(x='LOAN TYPE', y=feature, data=df_model_data, palette='viridis', hue='LOAN TYPE', legend=False)
    plt.title(f'Distribution of {feature} by Loan Type', fontsize=12)
    plt.xlabel('Loan Type', fontsize=10)
    plt.ylabel(feature, fontsize=10)
    plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

print("Distribution plots for top features by loan type generated.")

```



```
print("\nStep 10: Visualizing Actual vs. Predicted Probabilities...")

# Get a list of all unique 'LOAN TYPE' values
loan_types = df_cleaned['LOAN TYPE'].unique()
n_loan_types = len(loan_types)

# Determine the number of rows and columns for subplots
rows = int(np.ceil(n_loan_types / 2)) # For example, 2 plots per row
cols = 2 if n_loan_types > 1 else 1

plt.figure(figsize=(15, 6 * rows)) # Adjust figure size dynamically

for i, loan_type in enumerate(loan_types):
    ax = plt.subplot(rows, cols, i + 1)

    # Filter data for the current loan type
    loan_data = df_cleaned[df_cleaned['LOAN TYPE'] == loan_type].sort_values('QUARTER-END-DATE')

    # Plot historical GNPA Ratio
    sns.lineplot(x='QUARTER-END-DATE', y='GNPA_RATIO', data=loan_data, ax=ax, marker='o', color='skyblue')
    ax.set_title(f'{loan_type} - GNPA Ratio and Predicted Risk', fontsize=12)
    ax.set_xlabel('Quarter-End Date', fontsize=10)
    ax.set_ylabel('GNPA Ratio (%)', fontsize=10)

    # Retrieve predicted RISK_PROBABILITY for the current loan type
    predicted_risk = final_predictions[final_predictions['LOAN TYPE'] == loan_type]['RISK_PROBABILITY'].iloc[0]

    # Find the latest QUARTER-END-DATE for annotation
    latest_date = loan_data['QUARTER-END-DATE'].max()
    latest_gnpa = loan_data[loan_data['QUARTER-END-DATE'] == latest_date]['GNPA_RATIO'].iloc[0]

    # Annotate the plot with predicted RISK_PROBABILITY
    ax.annotate(f'Predicted Risk: {predicted_risk:.2%}',
                xy=(latest_date, latest_gnpa),
                xytext=(latest_date + pd.DateOffset(months=2), latest_gnpa + 5), # Adjust text position as needed
                arrowprops=dict(facecolor='red', shrink=0.05),
```

```
fontsize=10, color='red', ha='left', va='bottom')
```

```
# Rotate x-axis labels for better readability
ax.tick_params(axis='x', rotation=45)
ax.grid(True, linestyle='--', alpha=0.7)
```

```
plt.tight_layout()
plt.show()
```

```
print("Visualizations of actual vs. predicted probabilities generated for each loan type.")
```

Step 10: Visualizing Actual vs. Predicted Probabilities...

